

Search, Ground, Plan: Functional Sufficiency for Task and Motion Planning under Incomplete Scene Knowledge

Narendhiran Vijayakumar^{*1}, Nav Singhal¹, Girish Varma², Antony Thomas¹

¹Robotics Research Center, IIIT Hyderabad ²Center for Security, Theory and Algorithmic Research, IIIT Hyderabad

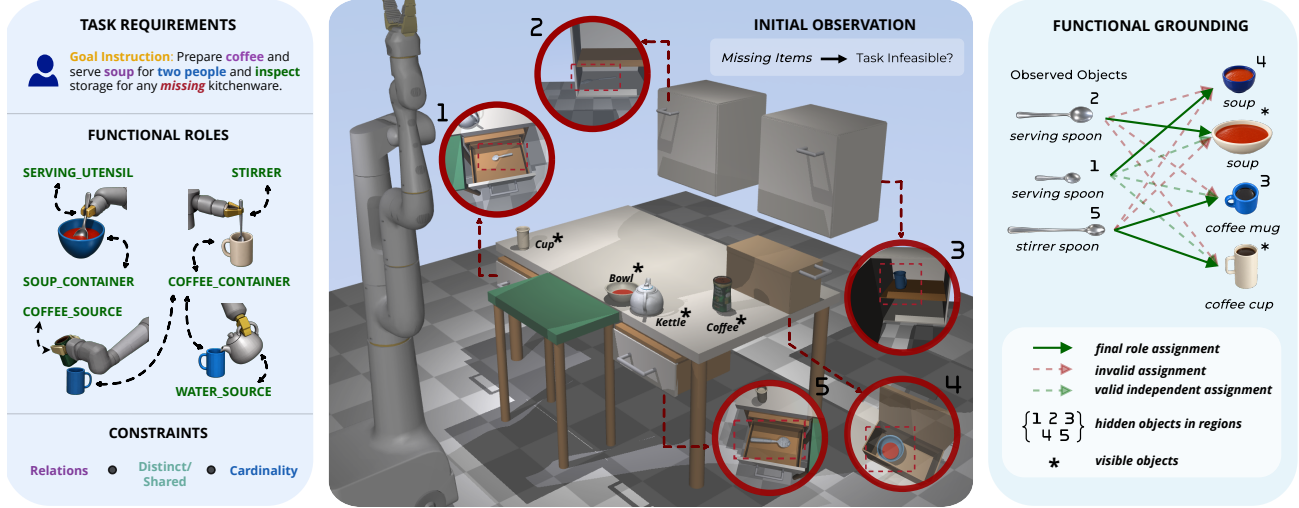


Fig. 1. A task and motion planning problem with incomplete object details, inter-object relations, and scene observations. GRAB-TAMP incorporates task-level functional requirements (left), searches for hidden task-relevant objects (center), and verifies joint role assignments (right) before planning.

Abstract— Foundation models (FMs) have expanded task and motion planning (TAMP) to manipulation problems specified through language and visual observations. However, incomplete scene knowledge leaves a critical gap between understanding what the task requires and knowing whether the physical scene can actually realize it. We introduce GRAB-TAMP, an FM-based TAMP framework that *searches* for scene entities required for task completion, *grounds* functional roles to valid physical objects, and *plans* only after a complete joint assignment establishes functional sufficiency. We represent the task through functional roles, relations, and assignment constraints, and incrementally inspect the scene while requirements remain unresolved, verifying candidate objects through semantic, geometric, and relational checks. We evaluate GRAB-TAMP across 32 scene variants spanning Kitchen, Living Room, and Workshop domains. Across 200 feasible trials, our approach achieves 54.0% end-to-end success with 67.3% plan goal coverage. Compared with three FM-based TAMP frameworks under the same execution setting, GRAB-TAMP improves end-to-end success by 25.7 percentage points over the mean baseline. Implementation and evaluation code: <https://anonymous.4open.science/r/GRAB-TAMP>.

Index Terms—Task and Motion Planning, Functional Grounding, Partial Observability

I. INTRODUCTION

Many everyday manipulation tasks appear trivial largely because humans implicitly reason about far more than what is explicitly stated in the instruction. An instruction specifies the intended outcome, but rarely describes every object, relation, or intermediate observation required to achieve it. Consider the scene in Fig. 1, where the robot is instructed to *prepare coffee and serve soup for two people*. The language instruction inherently specifies the functional requirements

for task completion, but leaves unresolved which physical scene objects satisfy them. This becomes challenging when the initial observation does not provide enough evidence to make that realization. For example, a suitable object may be occluded, contained within an articulated region, or simply outside the present view. Its absence from the current observation therefore does not imply that the task itself is infeasible.

At the same time, a new observation alone may be insufficient. A newly observed object may be irrelevant to the task, semantically suitable but geometrically incompatible, or satisfy both requirements. For example, a spoon may be recognized as an appropriate utensil but too large for the intended container, rendering it geometrically incompatible (see functional grounding block in Fig. 1). Thus, a valid functional assignment must satisfy both the semantic and geometric requirements of the role it fulfills.

Fig. 1 further illustrates why certain functional assignments must be considered jointly. A narrow spoon may fit both a small and a wide bowl, while a larger spoon may fit only the wide bowl. Assigning the narrow spoon to the wide bowl may thus appear locally valid but leave no feasible utensil for the small bowl. This shows why object–role assignments must be considered jointly rather than independently. We refer to this problem as **functional grounding**, assigning physical scene entities to task-level roles subject to the properties, relations, cardinalities, and interaction requirements imposed by the task. Whether an object constitutes a valid choice may depend on what has already been assigned in the task. Conversely, the absence of a valid assignment in the current

observation does not necessarily imply that the task itself is infeasible.

Given a functional grounding, a Task and Motion Planning (TAMP) framework [1] can reason about the discrete actions and the continuous motions required to execute them. Classical TAMP methods like PDDLStream [2], combines discrete decisions with continuous reasoning over grasps, placements, configurations, and motions. Recent embodied language [3] and multimodal models [4] have substantially improved the ability of robots to interpret abstract natural-language instructions, incorporate visual context, and reason over long-horizon tasks. Yet, under incomplete scene knowledge, a fundamental question remains unresolved: *whether the available physical entities are functionally sufficient to fulfill the task requirements before downstream planning and execution.*

This leaves a gap between interpreting task requirements and determining which scene entities satisfy those requirements for achieving *task feasibility*. In our framework, we represent the former as a fixed functional specification derived from the task, while the latter is resolved from the scene evidence accumulated thus far. When the currently observed objects are insufficient, additional scene information is acquired and newly revealed candidates are checked against the task requirements. Once a complete, valid object-role assignment is established, a TAMP planner can then verify task feasibility without repeatedly generating and evaluating partial plans.

To address the above challenges, we introduce GRAB-TAMP (*Grounded Role Assignment Bridging Incomplete Scene Knowledge and Task and Motion Planning*), a framework that bridges incomplete scene observations to downstream TAMP through functional grounding. GRAB-TAMP derives task-level functional requirements, incrementally acquires scene evidence when those requirements cannot yet be grounded, and jointly assigns verified physical entities to the required roles. An overview of the framework is shown in Fig. 2.

The contributions of this work are threefold:

- To our knowledge, this is the first foundation-model-guided TAMP framework to explicitly search for functionally required but currently unobserved entities and to use semantic and geometric verification to validate newly observed candidates.
- We formulate *functional grounding* under incomplete scene knowledge as a joint assignment between task-level roles and physical scene entities, capturing object-level requirements together with relational, cardinality, sharing, reuse, and distinctness constraints.
- We introduce *functional sufficiency* as an explicit criterion for determining when accumulated scene evidence is adequate for planning. Task planning proceeds only when all required functional roles admit a complete and jointly valid grounding to physical entities.

II. RELATED WORK

A. Foundation-Model-Assisted Task and Motion Planning

Classical TAMP approaches require an interface between the task planner and the motion planner. PDDLStream [2]

combines symbolic planning with streams that sample continuous quantities such as grasps, placements, and trajectories. However, this requires having the task domain, including its predicates and actions, together with the relevant objects and their relations in the scene, to be specified beforehand.

Foundation Models (FMs) have recently reduced the amount of high-level domain structure that must be provided manually. Large Language Models (LLMs) have been used to construct the planning representation directly from natural-language instructions. AutoTAMP [5] translates natural-language instructions into formal planning specifications, while CaStL [6] extracts task requirements from language and translates them into planner-checkable constraints. ViLaIn-TAMP [7] uses vision-language reasoning to construct a formal task representation. In these approaches, the FM is used primarily to bridge the gap between a task description and the structured representation required by downstream planning.

Other work uses FMs to guide the TAMP search. LLM³ [8] reasons from motion-planning failures to revise task-level decisions. Rather than waiting for such failures, VLM-TAMP [9] uses visually generated intermediate subgoals to reduce the effective planning horizon. ROBUST-TAMP [10] performs event-triggered replanning as new objects are observed or execution conditions change. VIZ-COAST [11] reasons about constraints associated with downward-refinement failures, while OWL-TAMP [12] uses VLM-generated symbolic and continuous constraints to support open-world manipulation. Together, these works show that FMs can influence several aspects of TAMP, from specifying and decomposing the problem to constraining, adapting, and repairing the search.

B. Planning with Incomplete Scene Knowledge

In TAMP, incomplete scene knowledge has largely been treated as uncertainty about the current state of the environment. Relevant objects may be hidden, their states may be unknown, and additional observations may therefore be required to determine whether a feasible plan can be synthesized.

Partial observability has been considered in TAMP through belief-space planning and explicit information-gathering actions [13]. Similarly, TAMPURA [14] models uncertainty over the initial state and action outcomes and plans over closed-loop controllers while reasoning about information-gathering actions and their associated risk. More recently, methods for TAMP under partial observability have combined task-level relevance reasoning with exploration so that objects required by the task can be progressively discovered [15]. These approaches generally assume that the task-relevant entities are already known, leaving the planner to reason about their location, state, or observability. In contrast, GRAB-TAMP first determines which physical scene entities can satisfy the functional roles required by the task, given RGB images and goal instruction.

FMs have also been used to reason about incomplete scene information. Zhao et al. [16] construct a symbolic belief representation using FM uncertainty estimates and plan strategic information-gathering actions when task-relevant

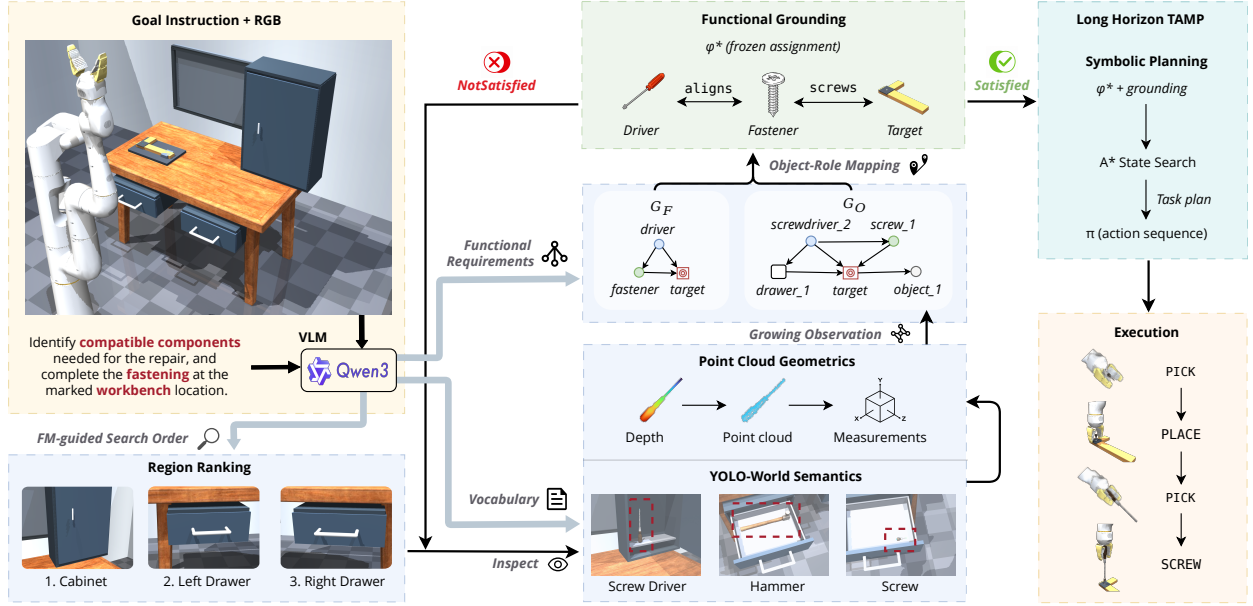


Fig. 2: GRAB-TAMP uses an FM to convert the goal and initial RGB observation into functional requirements G_F , a task-relevant perception vocabulary, and an inspection ranking. Scene observations incrementally update G_O^k , while semantic and geometric verification support joint object–role grounding to obtain ϕ^* . Once the scene is functionally sufficient, the grounded assignment and observed state instantiate the symbolic planner, which produces the executable action sequence. The complete framework is described in Section IV.

facts cannot be determined from the current observation. ROBUST-TAMP [10] plans incrementally as task-relevant objects are discovered, but cannot handle tasks requiring complete object knowledge before planning. CoCo-TAMP [17] introduces LLM commonsense knowledge into hidden-state estimation for partially observable TAMP. VAP-TAMP [18] employs active perception to gather additional information. These methods use FMs to reduce uncertainty or guide observation, but do not impose a pre-planning functional-sufficiency criterion based on complete joint role assignment. GRAB-TAMP instead evaluates whether observed entities are functionally sufficient before planning.

C. Functional Grounding and Object–Role Assignment

Object identification alone is insufficient for task execution; an object must also satisfy the physical requirements of its intended use. Prior work addresses this gap through affordance learning, object property prediction, and geometric grounding, enabling task-relevant properties to be inferred directly from scene observations by estimating affordances for previously unseen objects [19], grounding symbolic object references to physical scene instances [20], and predicting spatial affordances or interaction-relevant locations from visual observations [21]. However, these methods primarily ground individual objects, skills, predicates, or actions, whereas many tasks require jointly reasoning about multiple objects and their assignments. GRAB-TAMP addresses this gap by jointly grounding objects to task-level functional roles.

III. PROBLEM FORMULATION

A. Task and Scene Representation

We consider an environment containing a finite set of physical entities $\mathcal{O}$, including objects and placement regions and a finite set of inspectable regions $\mathcal{R}$. The robot initially observes only part of the environment through observation Z_0 . After a sequence of k measurements $(Z_{0:k})$, the objects

and the inspectable regions observed thus far are given by $\mathcal{O}_k \subseteq \mathcal{O}$, and $\mathcal{I}_k \subseteq \mathcal{R}$, respectively. Objects that have not yet been observed ($\mathcal{O} \setminus \mathcal{O}_k$) are treated as *unknown*, rather than assumed to be absent.

We denote by $Q = \{q_1, \dots, q_m\}$ the set of functional roles required by the task. A role describes what an object must be capable of doing, rather than the identity of the object expected to do it. Roles with cardinality greater than one are expanded into indexed role instances in Q , while $\mathcal{A}_F$ specifies the corresponding sharing, reuse, and distinctness constraints. For an observed object $o \in \mathcal{O}_k$, its compatibility with role q_i is written as $C_i(o) = C_i^{\text{sem}}(o) \wedge C_i^{\text{geom}}(o)$, combining semantic compatibility with the geometric properties needed to perform that function. Some requirements cannot be checked one object at a time. We therefore represent relations between objects assigned to different roles using functional edges

$$E_F \subseteq Q \times Q \times \mathcal{P} \quad (1)$$

where each edge specifies a predicate in $\mathcal{P}$ that must hold between the objects assigned to the corresponding pair of roles. For example, when $q_i \equiv \text{stirrer}$, $q_j \equiv \text{coffee-container}$, the corresponding compatibility predicate may evaluate to 1, whereas for $q_j \equiv \text{soup-container}$, it may evaluate to 0 (Fig. 1). Similarly, a task may also impose assignment constraints $\mathcal{A}_F$, for example, requiring two roles to use distinct objects or allowing the same object to be shared when the task permits it. The functional requirements are thus represented by

$$G_F = (Q, E_F) \quad (2)$$

together with the assignment constraints $\mathcal{A}_F$. Rather than specifying which particular objects should be used, we describe the task through the functions that objects need to perform. To this end, we define the task specification

$$\Psi = (G_F, \Gamma, \mathcal{V}_g, \mathcal{A}_F, \pi_R) \quad (3)$$

TABLE I: Components in task specification and functional grounding.

Component	Role in the framework
$\mathcal{V}_g$	Specifies what perception should look for.
π_R	Defines where the scene should be inspected next.
G_F	Represents the functional requirements of the task.
G_O^k	Stores the scene information observed up to step k .
ϕ	Maps functional roles to observed physical objects.
Φ_k	Contains the assignments that satisfy the functional requirements.
ϕ_k^*	Denotes the valid assignment selected for downstream planning.

where G_F captures the functional requirements of the task, Γ specifies relations to achieve, while grounding checks functional compatibility, $\mathcal{V}_g$ provides a task-conditioned vocabulary for perception, and π_R gives region inspection order. $\mathcal{V}_g$, π_R guide what the system looks for and where it searches, but do not determine the final object–role assignment. We maintain an observed graph

$$G_O^k = (\mathcal{O}_k, E_O^k) \quad (4)$$

containing the objects and relations observed up to step k . The distinction between the two graphs is useful here: G_F stays fixed and describes what the task needs, while G_O^k grows as more of the environment becomes visible. Functional grounding therefore determines whether the objects observed so far collectively satisfy the requirements encoded by G_F .

B. Functional Grounding

At inspection step k , a grounding is a mapping $\phi_k : \mathcal{Q} \rightarrow \mathcal{O}_k$, which assigns an observed entity to each functional role. A valid grounding must satisfy the unary requirements of every role,

$$C_i(\phi_k(q_i)) = 1, \quad \forall q_i \in \mathcal{Q} \quad (5)$$

as well as every relation encoded in E_F ,

$$p(\phi_k(q_i), \phi_k(q_j)) = 1, \quad \forall (q_i, q_j, p) \in E_F \quad (6)$$

Assignments must also satisfy the sharing, distinctness, and set-level constraints in $\mathcal{A}_F$. We collect all such assignments in

$$\Phi_k = \left\{ \phi : \mathcal{Q} \rightarrow \mathcal{O}_k \mid \begin{array}{l} C_i(\phi(q_i))=1, \forall q_i, \\ p(\phi(q_i), \phi(q_j))=1, \forall (q_i, q_j, p) \in E_F, \\ \phi \models \mathcal{A}_F \end{array} \right\} \quad (7)$$

The key aspect is that grounding is evaluated *jointly* across the assigned objects. An object may satisfy the requirements of a role in isolation, yet the resulting assignment may be invalid when the required relations between roles are considered. For example, in Fig. 1, “serving spoon 2” satisfies the serving role but is incompatible with the smaller soup bowl. Consequently, an FM-proposed candidate is not accepted solely because it is semantically preferred. The task-conditioned vocabulary $\mathcal{V}_g$ guides perception toward relevant objects, while membership in Φ_k is determined by the verified functional constraints above. Whenever $\Phi_k \neq \emptyset$, the grounding module selects a valid assignment $\phi_k^* \in \Phi_k$, which is then fixed for downstream planning. The variables are summarized in Table I.

C. Functional Sufficiency, Search, and Planning

We say that the currently observed scene is *functionally sufficient* when at least one complete valid grounding exists:

$$\text{Sufficient}(G_O^k, G_F) = \begin{cases} 1, & \Phi_k \neq \emptyset, \\ 0, & \Phi_k = \emptyset \end{cases} \quad (8)$$

Algorithm 1 GRAB-TAMP

Input: Task goal g , initial observation Z_0 , inspectable regions $\mathcal{R}$, symbolic domain $\mathcal{D}$.

Output: Executable task plan π , or FAILURE with reasoning (Δ).

```

1:  $(G_F, \Gamma, \mathcal{V}_g, \mathcal{A}_F, \pi_R) \leftarrow \text{FMPropose}(g, Z_0)$  // Functional
   specification and search guidance
2:  $G_O^0 \leftarrow \emptyset; \mathcal{I} \leftarrow \emptyset; Z \leftarrow Z_0; k \leftarrow 1$  // Persistent observed
   graph, Inspected regions, Initial observation
3: while true do
4:    $G_O^k \leftarrow \text{UPDATEOBSGRAPH}(G_O^{k-1}, Z, \mathcal{V}_g)$  // Add newly
   observed semantic and geometric evidence
5:    $\mathcal{C}^k \leftarrow \text{VERIFYCANDIDATES}(G_O^k, G_F)$  // Semantic,
   unary-geometric, and relational verification
6:    $\Phi^k \leftarrow \text{GROUNDROLES}(\mathcal{C}^k, G_F, \mathcal{A}_F)$  // Joint assign-
   ments satisfying roles, relations, and reuse constraints
7:   if  $\Phi^k \neq \emptyset$  then
8:      $\phi^* \leftarrow \text{SELECTGROUNDING}(\Phi^k)$  // Fix one verified
   complete assignment
9:      $P \leftarrow \text{BUILDSYMBOLICPROBLEM}(\mathcal{D}, G_O^k, G_F, \phi^*, \Gamma)$ 
   // Instantiate grounded task and scene state
10:     $\pi \leftarrow \text{ASTAR}(P)$  // Synthesize manipulation sequence
11:    if  $\pi \neq \text{INFEASIBLE}$  then
12:      return  $\pi$ 
13:    else
14:       $\Delta \leftarrow \text{PLANNINGFAILUREREASON}(G_O^k, G_F, \phi^*, \Gamma)$ 
15:      return  $(\text{PLANNINGFAILURE}, \Delta)$ 
16:    end if
17:  end if
18:  if  $\mathcal{I} = \mathcal{R}$  then
19:     $\Delta \leftarrow \text{GROUNDINGFAILUREREASON}(G_O^k, G_F, \mathcal{C}^k)$ 
20:    return  $(\text{GROUNDINGFAILURE}, \Delta)$ 
21:  end if
22:   $r \leftarrow \text{NEXTREGION}(\pi_R, \mathcal{R} \setminus \mathcal{I})$  // Follow the FM-provided
   inspection ordering
23:   $Z \leftarrow \text{INSPECTANDOBSERVE}(r)$  // New scene informa-
   tion
24:   $\mathcal{I} \leftarrow \mathcal{I} \cup \{r\}$ 
25:   $k \leftarrow k + 1$ 
26: end while

```

This gives the search process a simple stopping condition. If $\Phi_k \neq \emptyset$, the currently observed objects are sufficient for functional grounding and no additional inspection is needed. If $\Phi_k = \emptyset$ while uninspected regions remain, the next region is selected according to π_R , inspected, and incorporated into G_O^k . Grounding is then reconsidered over the expanded observation.

If all regions in $\mathcal{R}$ have been inspected and Φ_k remains empty, the task is declared *functionally infeasible* within the explored scene, since no assignment satisfying G_F could be found. Functional sufficiency, however, does not by itself guarantee that the manipulation task is executable. Once a valid assignment ϕ_k^* has been obtained, the abstract roles in G_F are replaced by their corresponding physical objects and, together with Γ , accumulated scene information, and a fixed symbolic action domain $\mathcal{D}$, are used to construct the downstream TAMP problem. If no plan exists despite having a valid functional grounding, the outcome is instead a *planning failure*. This keeps two different questions separate: whether suitable objects can be found and assigned to the required functions, and whether those grounded objects admit a feasible manipulation plan.

IV. METHODOLOGY

An overview of GRAB-TAMP is given in Algorithm 1. The framework builds the observed graph incrementally and evaluates the available objects against the fixed functional requirements. The following subsections describe how the functional graph G_F is generated, how G_O^k is updated, how candidates are verified and jointly grounded, and how the frozen grounding is compiled into the downstream planner.

A. FM Functional Specification and Search Guidance

Given the task instruction g and initial observation Z_0 , the FM provides the task specification Ψ (line 1). Rather than directly selecting the physical objects to be used, the FM first specifies the task-level roles and the properties and relations that must be verified for each role. The resulting specification is mapped onto the nodes and edges of G_F , together with the assignment constraints in $\mathcal{A}_F$, when the task requires objects to remain distinct, to be reused, or to be shared across roles. Candidate categories proposed for each role are collected into V_g and used only to guide open-vocabulary perception. They do not determine the final grounding. Whether an observed entity satisfies a role is decided later through the verification and joint assignment stages. The FM also provides the region ordering π_R , which determines the order in which currently uninspected regions are considered when additional scene information is required. In this way, the FM contributes to both the functional specification and the inspection guidance, where relevant objects may be found, as shown in Fig. 2.

B. Incremental Observed-Graph Construction

The initial observation Z_0 is processed first by the observed graph G_O^0 . Objects detected in the visible scene are added to G_O^0 together with their associated region and the available semantic and geometric evidence (line 4). The next uninspected region is selected by subroutine NEXTREGION according to π_R (line 22), after which INSPECTANDOBSERVE acquires the new observation (line 23). The resulting evidence is incorporated into G_O^k through subroutine UPDATEOBSGRAPH. Inspection of this region yields a new observation, from which newly visible objects and associated evidence are incorporated into G_O^k . Object attributes and relations can be extended as new evidence becomes available, but objects that have already been verified remain part of the graph after each inspection update.

C. Semantic and Geometric Verification

Candidate verification is performed by VERIFYCANDIDATES (line 5, Algorithm 1), which evaluates the semantic, unary geometric, and required pairwise checks described below. We use YOLO-World [22] with the task-conditioned vocabulary $\mathcal{V}_g$ to identify observed objects that are plausible candidates for each functional role. Objects that do not satisfy the corresponding semantic requirement C_i^{sem} are discarded for that role. The remaining candidates are then evaluated geometrically. For each object, its segmented depth observation is converted into object-level geometric evidence from which the required unary properties are evaluated. These checks instantiate C_i^{geom} for the corresponding role and include task-dependent properties such as usable dimensions, cavity structure, support geometry, or tool shape. Required pairwise constraints are then evaluated according to (6). For

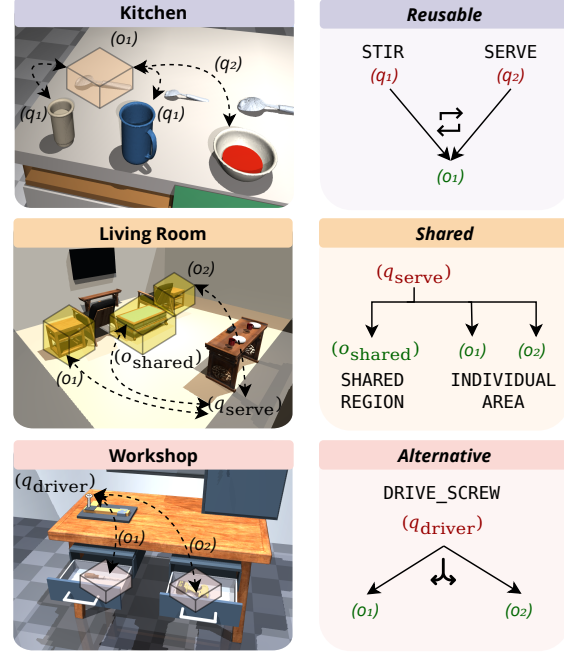


Fig. 3: Functional assignments need not be one-to-one. Objects may be reused, regions may be shared, and multiple candidates may satisfy the same task role, as formulated by $\mathcal{A}_F$ in (7).

each edge $(q_i, q_j, p) \in E_F$, the predicate p is evaluated on candidate entity pairs associated with roles q_i and q_j , retaining only pairs for which $p = 1$. This avoids constructing every possible geometric relation between every pair of objects in the scene. Thus, verification progressively narrows detected objects to semantically compatible candidates satisfying unary geometric requirements, before using pairwise relations to identify complete grounding.

D. Joint Object–Role Assignment

The verified candidates are passed to GROUNDROLES (line 6), which constructs the set of complete assignments Φ_k satisfying the role, relational, and assignment constraints. Candidate objects are not selected independently for each role, but rather together so that the required relations in E_F and the assignment constraints in $\mathcal{A}_F$ are satisfied by the same mapping. This matters when multiple individually valid candidates exist, since satisfying unary role constraints does not guarantee compatibility with other role assignments. Fig. 3 illustrates how the same grounding formulation accommodates reusable, shared, and alternative assignments across the three task domains. Distinct roles are prevented from taking the same physical instance, while roles that permit reuse may refer to an already assigned object when the remaining functional constraints are also satisfied. Shared assets are treated similarly, allowing one physical object to support multiple compatible parts of the task without requiring a duplicate instance.

If the current G_O^k does not provide any valid complete assignment, control returns to the inspection stage and the graph is expanded using the next region in π_R . Once $\Phi_k \neq \emptyset$, SELECTGROUNDING selects one verified mapping ϕ^* for downstream planning (line 8). At this point the object–role mapping is fixed. If no complete grounding exists after all

TABLE II: Task and scene definitions used for evaluating functional grounding.

Domain	Goal instruction	Functional relations	Hidden objects	Candidate regions	Variants F / I
<i>Kitchen</i>	Prepare and serve one coffee and one soup for each of two people. Make each coffee using coffee and water and stir it before serving. Serve each soup bowl with its own suitable eating utensil.	OPEN_CAVITY, ELONGATED_OBJECT, INSERTABLE_IN, and REACHES_BOTTOM.	Cups, bowls, stirring and eating utensils; required objects may be distributed across storage regions.	Countertop, drawers D1--D2, cabinets C1--C2, and bin B1.	6 / 6
<i>Living Room</i>	Prepare the living room for two people to enjoy refreshments while watching television. Provide each person with their own refreshment setting nearby, and place the entertainment control where it is accessible to both people.	PLANAR_SUPPORT, FITS_ON, FITS_SET_ON, and NEAR_SEAT.	None; task objects and placement regions are initially visible.	Left and right personal side tables and the shared coffee table.	6 / 4
<i>Workshop</i>	Identify the compatible components required to complete the fastening at the marked workbench location, complete the fastening, and leave any reusable equipment used for the task safely on the workbench.	GRASPABLE, FITS_TOOL_HEAD, FITS_IN, and NEAR_WORKPIECE.	Manual and power drivers, compatible Phillips screw, and wooden-hammer distractor.	Left drawer, right drawer, tool cabinet, and marked workbench target.	8 / 2

Note: F / I denotes feasible / infeasible variants.

regions are inspected, GROUNDINGFAILUREREASON reports the unresolved grounding conditions (line 19).

E. Grounded Symbolic Planning and Execution

The selected grounding ϕ provides the mapping between functional grounding and downstream planning, and replaces the abstract roles appearing in the task specification with the corresponding physical objects selected by ϕ . The accumulated scene information in G_O^k supplies the relevant object and region state, while Γ specifies the task-level conditions that the resulting plan must achieve. These inputs are passed to BUILDSYMBOLICPROBLEM (line 9), which constructs the grounded symbolic planning problem with action predicates. An A* search is then performed over this state space to obtain an ordered manipulation sequence: $\pi = [a_1, a_2, \dots, a_T]$.

The planner operates only after the functional roles have been resolved to physical objects. Scene inspection is therefore not included as a symbolic planning action, and the planner does not modify the frozen grounding ϕ^* . If no symbolic plan can be found after functional grounding, this is treated separately from the earlier case where no valid object-role assignment can be established from the explored scene. The resulting action sequence is passed to the execution layer, where each symbolic action is executed by the motion planner. If no symbolic plan is found despite a valid grounding, PLANNINGFAILUREREASON identifies the downstream planning failure (line 14), keeping this case distinct from grounding failure. This preserves a clear boundary between scene search and functional grounding on one side, and task planning and physical execution on the other.

V. EXPERIMENTATION

A. Environment Variants and Setup

All experiments are conducted in MuJoCo using the Google Robot mobile manipulator. We evaluate across three task domains—*Kitchen*, *Living Room*, and *Workshop*, comprising 32 scene variants in total: 20 feasible and 12 infeasible, each evaluated over 10 trials (see Table II). *Kitchen* includes hidden objects distributed across multiple storage regions, with varying cardinality and object reuse requirements; *Living Room* includes multiple placement and shared-region constraints with fully observable objects; and *Workshop* has alternative tools and hidden distractor objects. Each scene is observed through a fixed three-view camera configuration. We conduct 10 trials per scene variant, resulting in 320 trials for ablation and each baseline

TABLE III: End-to-end comparison of 3 task domains using Google Robot. Feasible and infeasible variants are reported separately.

Method	Feasible			All trials		Infeasible	
	N_F	PGC	E2E succ.	Avg. replans	N_I	NC	
<i>Kitchen</i>							
				<i>hidden across five storage regions</i>			
VLM-TAMP [9]	60	67.5	0.0	12.6	60	0.0	
OWL-TAMP [12]	60	45.5	0.0	0.0	60	13.3	
ROBUST-TAMP [10]	60	49.2	0.0	11.9	60	25.0	
GRAB-TAMP	60	82.8	65.0	–	60	50.0	
<i>Living Room</i>							
				<i>all objects initially visible</i>			
VLM-TAMP [9]	60	100.0	80.0	7.5	40	0.0	
OWL-TAMP [12]	60	100.0	60.0	0.0	40	5.0	
ROBUST-TAMP [10]	60	100.0	88.3	6.2	40	0.0	
GRAB-TAMP	60	48.7	31.7	–	40	70.0	
<i>Workshop</i>							
				<i>hidden alternative candidates</i>			
VLM-TAMP [9]	80	99.6	26.2	11.9	20	0.0	
OWL-TAMP [12]	80	100.0	15.0	2.62	20	0.0	
ROBUST-TAMP [10]	80	17.5	0.0	7.5	20	70.0	
GRAB-TAMP	80	69.6	62.5	–	20	20.0	
GRAB-TAMP (Overall)	200	67.3	54.0	–	120	51.7	

Note: PGC, E2E success, and NC are reported in %. Avg. replans is the mean number of planning re-involutions after the first, over all $N_F + N_I$ trials of a scene.

comparison. All inference calls use Qwen3.5–9B with full unquantized weights, served through vLLM on an inference server equipped with an NVIDIA RTX PRO 5000 Blackwell GPU card. All methods share the same budget, at most 15 model calls per trial, a 24,576-token generation ceiling and the checkpoint’s published sampling configuration beyond which we observed diminishing and saturated returns for the average of all baseline comparisons, including our work.

B. Evaluation Metrics

Feasible and infeasible variants are evaluated separately because they measure different behaviors. Let N_F and N_I denote the number of feasible and infeasible trials, respectively. For trial i , let G_i denote the set of required task goals and $\hat{G}_i$ the subset achieved by the generated plan. We define:

Plan Goal Coverage (PGC): For feasible tasks, we measure fraction of required task goals achieved by the generated plan

$$\text{PGC} = \frac{1}{N_F} \sum_{i=1}^{N_F} \frac{|G_i \cap \hat{G}_i|}{|G_i|} \times 100 \quad (9)$$

End-to-End (E2E) Success: Let $s_i \in \{0, 1\}$ indicate whether feasible trial i successfully satisfies all task goals after execution.

$$\text{E2E Success} = \frac{1}{N_F} \sum_{i=1}^{N_F} s_i \times 100 \quad (10)$$

Non-Commitment Rate (NC): Fraction of infeasible trials on which no plan is committed. It does not distinguish explicit infeasibility recognition from exhausted search or absent output,

$$NC = \frac{1}{N_I} \sum_{i=1}^{N_I} (1 - C_i) \times 100 = 100 - \text{Commit} \quad (11)$$

C. Overall Performance

Table III compares GRAB-TAMP against three FM-assisted TAMP approaches under the same execution settings. Across the 200 feasible trials, GRAB-TAMP achieves 54.0% E2E success, compared with 34.5% for VLM-TAMP. VLM-TAMP attains higher PGC (90.1% versus 67.3%) and its substantially larger PGC-to-E2E gap indicates that partial plan coverage often does not translate to complete execution. The difference between PGC and complete success is important in this setting, since several methods can produce partial plans of a long-horizon task without resolving all requirements needed for complete execution.

The baseline methods differ mainly in where FM reasoning enters the planning pipeline and how TAMP is organized around it. VLM-TAMP achieves substantial PGC in *Kitchen* (67.5%) and *Workshop* (99.6%), but only translates to 0% and 26.2% E2E success, respectively. Its intermediate-subgoal guidance can therefore support partial task progress once suitable objects are available, but does not explicitly determine whether all hidden or geometrically constrained objects required for complete execution have been resolved before planning. With replanning enabled, OWL-TAMP achieves 100.0% PGC and 15.0% E2E success in *Workshop*, averaging 2.62 replans across all trials. Despite complete plan goal coverage, execution success remains limited. In *Kitchen*, one planning cycle consumes the entire 15 call budget, leaving no room for a second and reducing the protocol to single-shot. ROBUST-TAMP performs best among the baselines in *Living Room*, reaching 88.3% E2E, but its performance falls sharply in *Kitchen* and *Workshop*. Although its event-triggered replanning allows the system to react to newly observed objects and execution changes, the resulting 49.2% and 17.5% PGC in these two domains indicate that reactive replanning alone does not resolve joint functional compatibility required when several hidden candidates must be considered together.

On the other hand, GRAB-TAMP shows its largest gains in *Kitchen* and *Workshop*, achieving 65.0% and 62.5% E2E success, respectively, where task completion depends on discovering hidden objects or geometrically constrained functional candidates. Its 31.7% success in *Living Room* is lower than all three baselines, suggesting that grounding ambiguities can arise when relatively simple, fully observable tasks are decomposed into a richer set of functional roles than is necessary for execution. Across the 120 infeasible trials, GRAB-TAMP obtains a 51.7% NC rate, compared with 24.2% and 0.0% for ROBUST-TAMP and VLM-TAMP, respectively. These rates do not always reflect explicit infeasibility recognition. For the baselines, any trial producing no executable plan counts as a noncommitment, conflating exhaustion with abstention. ROBUST-TAMP’s 70.0% in *Workshop* is due to replanning budget exhaustion without

TABLE IV: Verification and inspection ablations.

Verification Ablation					
Scene	Verification	Trials	Matching G _O (%)	Mismatching G _O (%)	Missing G _O (%)
<i>Kitchen</i>	Semantic only	60	96 (12.3)	19 (2.4)	665 (85.3)
	+ Unary	60	103 (13.2)	21 (2.7)	656 (84.1)
	+ Binary	60	514 (65.9)	63 (8.1)	203 (26.0)
<i>Living Room</i>	Semantic only	60	141 (18.1)	1 (0.1)	638 (81.8)
	+ Unary	60	96 (12.3)	3 (0.4)	681 (87.3)
	+ Binary	60	439 (56.3)	20 (2.6)	321 (41.2)
<i>Workshop</i>	Semantic only	80	86 (17.9)	13 (2.7)	381 (79.4)
	+ Unary	80	47 (9.8)	8 (1.7)	425 (88.5)
	+ Binary	80	239 (49.8)	55 (11.5)	186 (38.8)
Overall	Semantic only	200	323 (15.8)	33 (1.6)	1684 (82.5)
	+ Unary	200	246 (12.1)	32 (1.6)	1762 (86.4)
	+ Binary	200	1192 (58.4)	138 (6.8)	710 (34.8)

Inspection-Order Ablation				
Scene	Order	Regions	Checks	Time (s)
<i>Kitchen</i>	Fixed order	4.33	134.6	82.94
	FM-ranked	3.91	134.6	76.22
<i>Living Room</i>	Fixed order	0.00	23.2	6.77
	FM-ranked	0.00	23.2	6.77
<i>Workshop</i>	Fixed order	2.58	14.2	287.64
	FM-ranked	2.32	13.4	268.20

emitting an action sequence; OWL-TAMP records 0.0% NC in *Workshop*, committing to a partial action sequence in every infeasible trial before the budget is exhausted, and appears better suited to shorter-horizon tasks under a fixed model-call budget. GRAB-TAMP instead retains the source of failure across grounding and planning, distinguishing an unresolved functional assignment from a downstream planning failure.

D. Ablation

We perform two ablations as shown in Table IV. First, we evaluate how semantic, unary geometric, and binary relational checks affect the resulting functional grounding. Second, we evaluate whether using the FM-derived region ranking improves the efficiency of scene inspection relative to a fixed inspection order. Semantic verification uses YOLO-World with the FM-derived vocabulary, unary verification adds object geometry, and binary verification checks relations between candidates. For example, a spoon is useful only if it fits or reaches the container bottom. These checks determine which nodes and relations are retained in G_O^k without changing the joint-assignment procedure. Semantic and unary verification provide only 15.8% and 12.1% correct grounding evidence, while binary relations increase this to 58.4% and reduce missing evidence from 82.5% to 34.8%. Unary geometry applies a stricter filter and may reject semantically plausible candidates under noisy or incomplete geometric evidence, explaining the lower coverage before binary verification substantially improves joint assignment coverage. We also compare the FM-derived ranking π_R with a fixed order, keeping the remaining pipeline unchanged. In *Kitchen*, FM ranking reduces inspected regions from 4.33 to 3.91 and runtime from 82.94 s to 76.22 s; in *Workshop*, the corresponding values decrease from 2.58 to 2.32 regions, 14.2 to 13.4 candidate checks, and from 287.64 s to 268.20 s. *Living Room* is unchanged since all task-relevant entities are initially visible. Thus, FM ranking does not modify the grounding criterion, but reduces unnecessary scene inspection and reaches functional sufficiency with less search effort.

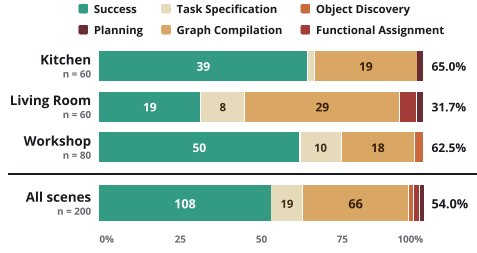


Fig. 4: Failure breakdown of GRAB-TAMP for feasible variants. Categories give the earliest stage that failed, not the point of termination: only Task Specification precludes a plan (PGC = 0), while others retain partial plan and contribute nonzero goal coverage.

VI. CONCLUSION AND DISCUSSION

We propose functional sufficiency as an explicit criterion for determining when the available scene evidence is adequate for downstream TAMP, and use functional grounding in GRAB-TAMP to establish this condition before planning while distinguishing unresolved grounding from downstream planning failure. The ablations further show that semantic verification alone is insufficient for functional grounding, while geometric and relational verification improve assignment validity and FM-ranked inspection reduces unnecessary search and inspection time. Failures can still arise when the FM omits or misstates a required role, relation, cardinality, or binding constraint, or when perception provides incorrect geometric evidence, as shown in the figure breakdown in Fig. 4. Misstated roles may remain unresolved; omitted requirements introduce no goals into Γ . From baseline comparisons, we find that end-to-end failure does not necessarily imply failed functional grounding, since partial task progress may still be achieved before all task requirements are satisfied. A practical limitation is that equivalent functional relations may be expressed differently across FMs and prompts, while the execution layer requires a fixed predicate vocabulary. We address this by mapping free-form relation descriptions to corresponding executable property checks, by using a zero-shot NLI DeBERTa [23] model. Further, a functionally valid assignment does not guarantee downstream motion feasibility. Planning or execution may still fail because of reachability or other manipulation constraints, and geometric verification is limited by the accuracy of the estimated scene geometry.

REFERENCES

- [1] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez, “Integrated task and motion planning,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 4, pp. 265–293, 2021.
- [2] C. R. Garrett, T. Lozano-Pérez, and L. P. Kaelbling, “PDDLStream: Integrating symbolic planners and blackbox samplers via optimistic adaptive planning,” in *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, vol. 30, 2020, pp. 440–448.
- [3] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, “Language models as zero-shot planners: Extracting actionable knowledge for embodied agents,” in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 162. PMLR, 2022, pp. 9118–9147.
- [4] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, T. Jackson, N. Brown, L. Luu, S. Levine, K. Hausman, and B. Ichter, “Inner monologue: Embodied reasoning through planning with language models,” in *Proceedings of the 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 205. PMLR, 2023, pp. 1769–1782.
- [5] Y. Chen, J. Arkin, C. Dawson, Y. Zhang, N. Roy, and C. Fan, “AutoTAMP: Autoregressive task and motion planning with LLMs as translators and checkers,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 6695–6702.
- [6] W. Guo, Z. Kingston, and L. E. Kavraki, “CaStL: Constraints as specifications through LLM translation for long-horizon task and motion planning,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 11957–11964.
- [7] J. Siburian, K. Shirai, C. C. Beltran-Hernandez, M. Hamaya, M. Görner, and A. Hashimoto, “Grounded vision-language interpreter for integrated task and motion planning,” *arXiv preprint arXiv:2506.03270*, 2025.
- [8] S. Wang, M. Han, Z. Jiao, Z. Zhang, Y. N. Wu, S.-C. Zhu, and H. Liu, “LLM³: Large language model-based task and motion planning with motion failure reasoning,” in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024, pp. 12086–12092.
- [9] Z. Yang, C. Garrett, D. Fox, T. Lozano-Pérez, and L. P. Kaelbling, “Guiding long-horizon task and motion planning with vision language models,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 16847–16853.
- [10] P. Ojha, N. Vijayakumar, N. Singhal, G. Varma, and A. Thomas, “Plan along the way: Event-triggered foundation-model planning for tamp execution in partially observable manipulation,” *arXiv preprint arXiv:2608.28075*, 2026.
- [11] M. Yan, M. Mengdibayev, A. Floros, W. Guo, L. E. Kavraki, and Z. Kingston, “Using VLM reasoning to constrain task and motion planning,” *arXiv preprint arXiv:2510.25548*, 2025.
- [12] N. Kumar, W. Shen, F. Ramos, D. Fox, T. Lozano-Pérez, L. P. Kaelbling, and C. R. Garrett, “Open-world task and motion planning via vision-language model generated constraints,” *IEEE Robotics and Automation Letters*, vol. 11, no. 3, pp. 3366–3373, 2026.
- [13] L. P. Kaelbling and T. Lozano-Pérez, “Integrated task and motion planning in belief space,” *The International Journal of Robotics Research*, vol. 32, no. 9–10, pp. 1194–1227, 2013.
- [14] A. Curtis, G. Matheos, N. Gooskar, V. Mansinghka, J. B. Tenenbaum, T. Lozano-Pérez, and L. P. Kaelbling, “Partially observable task and motion planning with uncertainty and risk awareness,” in *Robotics: Science and Systems (RSS)*, 2024.
- [15] Y. Ma, Y. Yuan, S. Wu, and H. Yuan, “A task and motion planning framework for partially observable household manipulation scenes,” *Advanced Intelligent Systems*, 2025.
- [16] L. Zhao, W. McClinton, A. Curtis, N. Kumar, T. Silver, L. P. Kaelbling, and L. L. S. Wong, “Seeing is believing: Belief-space planning with foundation models as uncertainty estimators,” *arXiv preprint arXiv:2504.03245*, 2025.
- [17] Y. Kim, R. Arora, R. Martin-Martin, P. Stone, B. Abbatematteo, and Y. Sung, “Large-language-model-guided state estimation for partially observable task and motion planning,” *arXiv preprint arXiv:2603.03704*, 2026.
- [18] A. Oloo, Z. Altaweel, Y. Hayamizu, P. Liu, Y. Ding, S. Amiri, H. Yang, A. Kaminski, C. Esselink, C. Paxton, X. Zhang, and S. Zhang, “Robot planning and situation handling with active perception,” *arXiv preprint arXiv:2604.26988*, 2026.
- [19] A. Curtis, X. Fang, L. P. Kaelbling, T. Lozano-Pérez, and C. R. Garrett, “Long-horizon manipulation of unknown objects via task and motion planning with estimated affordances,” in *2022 IEEE International Conference on Robotics and Automation (ICRA)*, 2022.
- [20] Y. Ding, X. Zhang, X. Zhan, and S. Zhang, “Learning to ground objects for robot task and motion planning,” *IEEE Robotics and Automation Letters*, 2022.
- [21] W. Yuan, J. Duan, V. Blukis, W. Pumacay, R. Krishna, A. Murali, A. Mousavian, and D. Fox, “RoboPoint: A vision-language model for spatial affordance prediction in robotics,” in *Proceedings of the 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270. PMLR, 2025.
- [22] T. Cheng, L. Song, Y. Ge, W. Liu, X. Wang, and Y. Shan, “Yolo-world: Real-time open-vocabulary object detection,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 16901–16911.
- [23] M. Laurer, W. van Atteveldt, A. Casas, and K. Welbers, “Building efficient universal classifiers with natural language inference,” Dec. 2023.